

W3Schools Css Flexbox

CSS Flexbox ? Flexible Box Layout Module

Fuente: https://www.w3schools.com/css/css3_flexbox.asp

DESCRIPCIÓN

CSS Flexbox (Flexible Box Layout) es un modelo de layout para disponer items (horizontal o verticalmente) dentro de un contenedor, de manera flexible y responsive.

Flexbox facilita el diseño de layouts sin necesidad de usar float o position.

FLEXBOX vs GRID

CSS Flexbox ? Layout en UNA dimensión (filas OR columnas)

CSS Grid ? Layout en DOS dimensiones (filas AND columnas)

COMPONENTES DE FLEXBOX

1. Flex Container ? El elemento padre con display: flex o display: inline-flex
2. Flex Items ? Los hijos directos del flex container (son flex items automáticamente)

ACTIVAR FLEXBOX

```
.container {  
  display: flex;  
}  
  
/* O inline */  
.container {  
  display: inline-flex;  
}
```

PROPIEDADES DEL FLEX CONTAINER

flex-direction

Dirección de los flex items.

```
.container { flex-direction: row; }          /* default: izquierda a derecha */  
.container { flex-direction: row-reverse; }  /* derecha a izquierda */  
.container { flex-direction: column; }       /* arriba a abajo */  
.container { flex-direction: column-reverse; }/* abajo a arriba */
```

flex-wrap

Permitir o no que los items salten a la siguiente línea.

```
.container { flex-wrap: nowrap; }           /* default: todo en una línea */  
.container { flex-wrap: wrap; }             /* salto de línea cuando no cabe */  
.container { flex-wrap: wrap-reverse; }
```

justify-content

Alineación en el eje principal (horizontal si flex-direction: row).

```
.container { justify-content: flex-start; }  /* default: inicio */  
.container { justify-content: flex-end; }    /* final */  
.container { justify-content: center; }      /* centrado */  
.container { justify-content: space-between; } /* espacio entre items */  
.container { justify-content: space-around; } /* espacio alrededor */  
.container { justify-content: space-evenly; } /* espacio igual */
```

align-items

Alineación en el eje secundario (vertical si flex-direction: row).

```
.container { align-items: stretch; }      /* default: llena el alto */
.container { align-items: flex-start; }    /* alineado al inicio */
.container { align-items: flex-end; }      /* alineado al final */
.container { align-items: center; }        /* centrado */
.container { align-items: baseline; }      /* alineado a la línea de texto */
```

align-content

Alineación de múltiples filas (solo cuando hay wrap).

```
.container { align-content: stretch; }
.container { align-content: flex-start; }
.container { align-content: flex-end; }
.container { align-content: center; }
.container { align-content: space-between; }
.container { align-content: space-around; }
```

gap

Espacio entre flex items (desde CSS Grid, también funciona en Flexbox).

```
.container { gap: 20px; }                 /* gap igual horizontal y vertical */
.container { gap: 10px 20px; }            /* row-gap col-gap */
.container { row-gap: 10px; }
.container { column-gap: 20px; }
```

PROPIEDADES DE LOS FLEX ITEMS

order

Orden visual del item (default: 0).

```
.item-1 { order: 1; } /* aparece al final */
.item-2 { order: -1; } /* aparece al inicio */
```

flex-grow

Cuánto puede crecer un item relativo a los otros (default: 0, no crece).

```
.item { flex-grow: 1; } /* todos crecen igual */
.item-main { flex-grow: 2; } /* este crece el doble que los otros */
```

flex-shrink

Cuánto puede encoger un item (default: 1, puede encoger).

```
.item { flex-shrink: 0; } /* no encoge */
```

flex-basis

Tamaño inicial del item antes de distribuir espacio.

```
.item { flex-basis: 200px; } /* parte de 200px */
.item { flex-basis: 30%; } /* parte del 30% */
.item { flex-basis: auto; } /* default: tamaño del contenido */
```

flex (shorthand)

Combina flex-grow, flex-shrink y flex-basis.

```
.item { flex: 1; } /* = flex: 1 1 0% */
.item { flex: 1 1 auto; } /* grow=1, shrink=1, basis=auto */
.item { flex: 0 0 200px; } /* tamaño fijo 200px */
.item { flex: 1 1 300px; } /* crece y encoge, parte de 300px */
```

align-self

Sobreescribe align-items para un item individual.

```
.item { align-self: auto; }  
.item { align-self: flex-start; }  
.item { align-self: flex-end; }  
.item { align-self: center; }  
.item { align-self: stretch; }
```

EJEMPLOS PRÁCTICOS

Centrar horizontal y verticalmente:

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 100vh;  
}
```

Barra de navegación (links a la derecha):

```
.navbar {  
  display: flex;  
  justify-content: space-between; /* brand izq, links der */  
  align-items: center;  
  padding: 10px 20px;  
}
```

Grid de cards responsive (sin Bootstrap):

```
.cards-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 24px;  
  justify-content: flex-start;  
}  
.card {  
  flex: 1 1 280px; /* crece, encoge, parte de 280px */  
  min-width: 280px;  
  max-width: 400px;  
}
```

Columnas iguales:

```
.row {  
  display: flex;  
  gap: 16px;  
}  
.col {  
  flex: 1; /* todos iguales */  
}
```

Footer siempre abajo (sticky footer):

```
body {  
  display: flex;  
  flex-direction: column;  
  min-height: 100vh;  
}
```

```
main {
  flex: 1;          /* main crece para llenar el espacio */
}
footer {
  flex-shrink: 0;   /* footer no encoge */
}
```

COMBINADO CON MEDIA QUERIES

```
/* Mobile: columna */
.container {
  display: flex;
  flex-direction: column;
}

/* Tablet y desktop: fila */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}
```

FLEXBOX vs. BOOTSTRAP GRID

	Flexbox (CSS puro)	Bootstrap Grid (.col-*)
Dimensiones	1D (fila o columna)	1D (siempre fila)
Columnas	Flexible (basado en flex)	12 columnas fijas
Código	Más CSS, control total	Más clases HTML, más rápido
Responsive	Manual con media queries	Automático con breakpoints
Para TP1	styles.css custom	Estructura principal HTML

RECOMENDACIÓN PARA TP1

Usar Bootstrap Grid para la estructura principal (Navbar, grid de cards, forms)
y Flexbox en styles.css para ajustes finos y layouts custom dentro de los componentes.